V-Trace: Final Project Report

Multi-Attribute Vehicle Re-Identification System

Gurneet Kaur Bhuller (A0354164E) · Group 21

Graduate Certificate in Pattern Recognition Systems · Practice Module

31 October 2026

Executive Summary

Finding one vehicle across several traffic or dash-cam videos is slow: an investigator usually starts from a description ("a red SUV, plate starting SGX…") and the plate is often unreadable. **V-Trace** turns short videos into searchable **sightings** (every vehicle detected, followed while in view and described), returns a **ranked list of best matches** for a description, and, once the user selects the right one, shows **every camera the vehicle was seen in, with the camera's location, the real-world date and time, and a frame from each sighting**.

V-Trace is built entirely from **published, pretrained models** – YOLO11 and ByteTrack, CLIP, EasyOCR and the **FastReID model trained on VeRi-776 by the VeRi authors' team** – and nothing was trained for this project. The Re-ID model reproduces its published accuracy on VeRi-776 (**Rank-1 97.0%, mAP 81.6%**; published 97.0% / 81.9%). On real street footage it is much weaker (Rank-1 36%), so the decision rule matters: a ratio test and a real-world time window raise the share of correct cameras reported from 26% to **81%**, without training.

1. Problem and Objectives

Road cameras and dash cams record most city streets, but after an incident (a hit-and-run, a stolen car) the search is largely manual. Three things make it hard: **the plate is often unreadable** (in our demo footage, no plate was readable); **the search starts from a description**, not an image; and **each camera sees the car briefly, from its own angle**, while different cars of the same model can look almost identical.

Objective. A basic, working vehicle Re-ID platform that (1) takes demo or uploaded videos, one video per camera; (2) groups the frames in which a vehicle is seen continuously into one sighting; (3) ranks sightings against a description (colour, type, plate, distinctive features, free text); and (4) for the selected sighting, shows every camera it was seen in with location, real-world time and a frame. The output is a ranked list for a person to review, never an identity claim. Live camera networks, owner records and training new networks are out of scope.

Measure	Target	Result
Re-ID model on VeRi-776 (official split), Rank-1 / mAP	within 1 point of 97.0 / 81.9	97.0 / 81.6 ✓
Same-vehicle threshold: true / wrong pairs passing (VeRi-776)	≥ 85% / ≤ 1%	89% / 0.8% ✓
Demo footage: cameras reported that are correct	≥ 70%	81% ✓
Demo footage: queries wrongly placed on the distractor road	≤ 10%	0% ✓
Demo vehicle found from its description	top 5	#3 ✓
Upload processing time (Fast), 20–22 s clips, laptop CPU	≤ 1.5 × clip length	about 1× ✓

2. Tools and Techniques

Step	Tool / model (pretrained, used as published)	Technique
Detect vehicles	**YOLO11n** (Ultralytics), COCO classes car, motorcycle, bus, truck	Single-stage CNN object detection
Follow a vehicle in one video	**ByteTrack** (Ultralytics)	Tracking-by-detection; also links low-confidence boxes, so tracks survive partial occlusion
Same vehicle in another camera	**FastReID SBS(R50-ibn)** trained on VeRi-776 (JDAI-CV)	Supervised metric learning: ResNet-50 with instance-batch normalisation and non-local blocks, GeM pooling, BN-neck; cosine similarity of 2048-d embeddings
Colour, type, features, free text	**CLIP ViT-B/32** (OpenAI, open_clip)	Zero-shot vision-language scoring of image vs text prompts
Licence plate	**EasyOCR** (JaidedAI)	OCR with wildcard (?) edit-distance matching
Decide "seen in this camera"	our decision rule	Similarity threshold + ratio test (Lowe, 2004) + real-world time window
Application	**Streamlit**, OpenCV, PyTorch (CPU)	Web app with admin sign-in; tests with pytest and Streamlit AppTest

Why these choices. VeRi-776 (Liu et al., 2016) is the standard vehicle Re-ID benchmark, and FastReID (He et al., 2020) is the toolbox released by the same research team with trained VeRi-776 models; using it avoids training and gives far stronger matching than generic embeddings (an earlier version of this project with ImageNet ResNet-50 and CLIP embeddings plus a trained LDA metric and Random Forest reached at most about 77% Rank-1 on VeRi). CLIP needs no labelled data and also handles free text. YOLO11 and ByteTrack are fast enough on a CPU for upload-and-search.

3. System Design and Models

```
Videos ─► YOLO11n detection ─► ByteTrack tracking ─► sightings ─► descriptors ─► vehicle index
                                                                                      │
            FastReID VeRi-776 embedding · CLIP zero-shot colour/type/features · EasyOCR plate
                                                                                      │
Description ─► ranked best matches ─► user SELECTS one ─► same vehicle in every camera ─► location + real time + frame
                                                  ▲
              camera details: location + recording start  (real-world time = recording start + position in the video)
```

3.1 Videos → sightings. Every 3rd frame (every 5th with *Fast processing*) goes through YOLO11n and ByteTrack links one vehicle's detections. One track is one **sighting**: its time span, boxes, four good crops spread in time (large, confident, not cut by the frame edge) and one full frame with the vehicle boxed. Tracks with fewer than 4 detections are dropped.

3.2 Camera details and real-world time. A video knows only positions in the video, so each camera has a location and a **recording start**; a sighting's real-world time is the recording start plus its position in the video, so cameras can be seconds, minutes or hours apart, and the route is ordered by real-world time. Uploads ask for these next to each file and the *Cameras* page edits them. The demo videos carry no such metadata, so their details are **curated** and placed in Singapore: camA–camC on Orchard Road around the Scotts Road junction (same clock, Wed 28 Oct 2026, 08:15:00) and camD on Bukit Timah Road near Newton Circus (08:17:30).

3.3 Describing each sighting. Each crop goes through the FastReID network (256×256 input) and the embeddings are averaged and L2-normalised; `vtrace/fastreid.py` only re-declares the network so the published checkpoint loads with plain PyTorch. CLIP compares the averaged crop embedding with prompts such as "a photo of a red car" (ten colours and nine types from VeRi), and scores distinctive features ("a car with a rear spoiler") and free text against "a photo of a car". EasyOCR reads the lower part of the two largest crops and keeps only confident readings.

3.4 Description → ranked matches. Each field the user fills in gives every sighting a 0–1 score; the final score is the weighted mean over **only the fields given** (plate 0.35, colour 0.30, type 0.25, features 0.10, free text 0.10), so a missing field never counts against a vehicle. Unread plates get a neutral 0.3.

3.5 Selected match → where it was seen. For every other camera, the most similar sighting (cosine of FastReID embeddings) is reported as the same vehicle only if (1) its similarity is at least **0.40**, chosen from the VeRi-776 score distributions; (2) it beats the camera's second-best sighting by **0.08** (ratio test: look-alikes are left to the user); and (3) with the time limit on, it is within **10 s** of the selected sighting in real-world time. For cameras with no confident match, the three closest vehicles are shown instead.

3.6 Application. After an admin sign-in, the **Search** page has three steps: *Describe the vehicle*, *Best matches* (crop, camera, real-world time, location, score, **Select**) and *Where this vehicle was seen* (route with real-world times and the time between cameras, one row per sighting with location, date and time, crop and frame, and a map). The **Cameras** page holds each camera's details and the matching settings. Demo cameras are bundled pre-processed, and seven upload-ready test clips (`test_videos/`) come with their camera details and expected results.

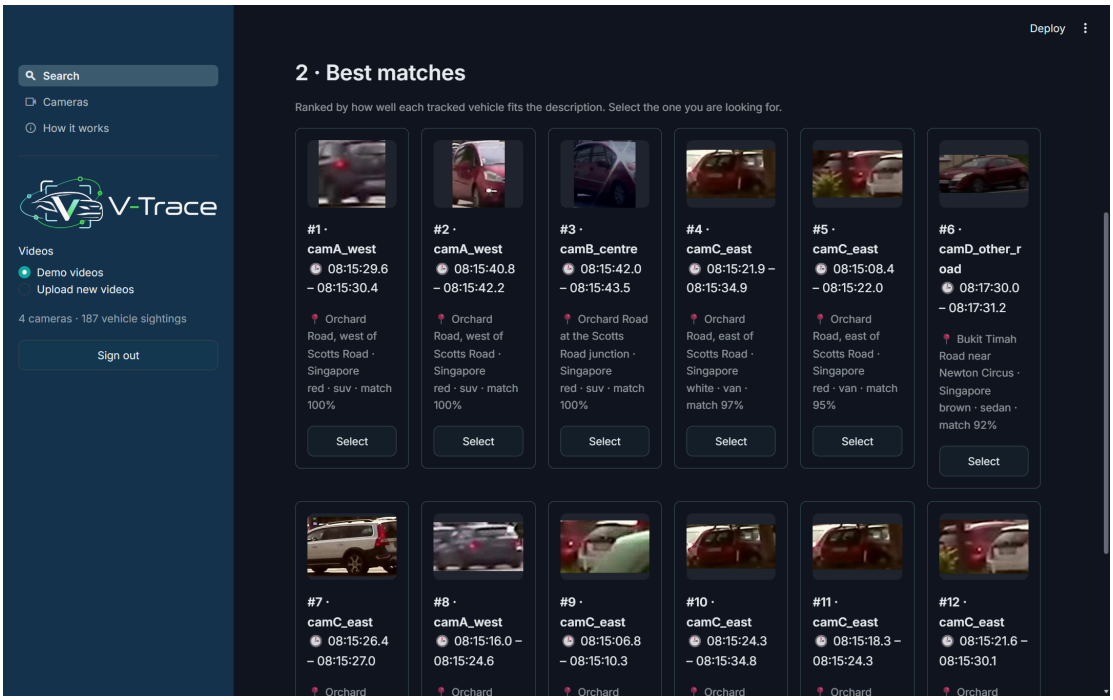


Figure 1 – best matches for the description "red SUV"

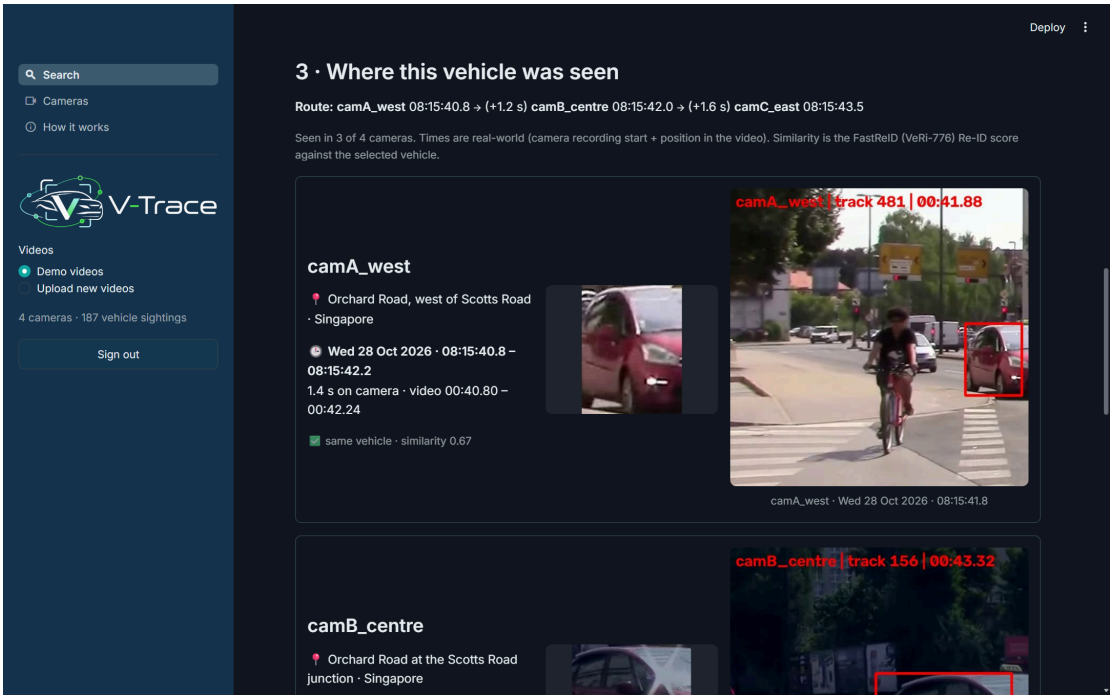


Figure 2 – where the selected vehicle was seen: location, real-world time and a frame per sighting

4. System Performance

4.1 Re-ID model on VeRi-776 (`scripts/eval_veri.py`). Official split: 1,678 query images ranked against 11,579 test images; same-vehicle, same-camera gallery images ignored. Nothing was trained.

Metric	Our run	FastReID model zoo
Rank-1	97.0%	97.0%
Rank-5	98.6%	–
mAP	81.6%	81.9%

Same-vehicle pairs from different cameras have a median similarity of 0.62 (10th percentile 0.39); 99% of different-vehicle pairs score below 0.38. At 0.40, 89% of true pairs and 0.8% of wrong pairs pass.

4.2 Demo cameras (`scripts/demo_eval.py`). A 46 s, 1080p street recording (Wikimedia Commons, CC BY 3.0) is cut into three non-overlapping zones along the road, each with its own exposure and colour cast, so a car passes A → B → C or the reverse; a fourth camera shows another road (Figure 3). Processing gives 187 sightings. Tracking the uncropped clip once and mapping each sighting back onto it by box overlap gives a reference identity per sighting (*pseudo* ground truth); 19 vehicles appear in two or more cameras and 7 in all three. Over 55 queries, the correct sighting in another camera is ranked first for **36%** (Rank-5 58%, mAP 38%).

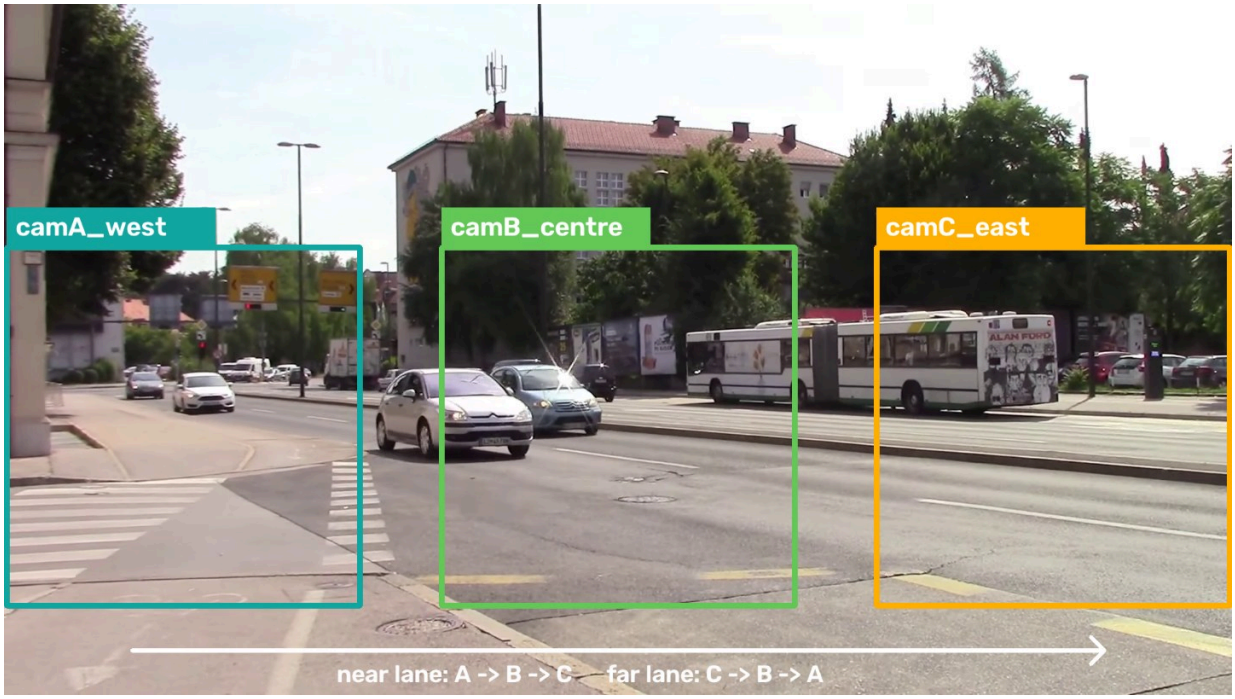


Figure 3 – the three demo camera zones cut from one street recording

4.3 Decision-rule ablation.

Rule for "seen in this camera"	Cameras reported correct	Cameras found	Queries placed on the other road
Best sighting above 0.40	26%	44%	73%
+ ratio test	53%	22%	16%
+ real-world time window, 10 s (final)	**81%**	32%	0%

The ratio test doubles precision by refusing ambiguous cameras; the time window removes impossible matches. Part of the window's gain comes from the curated clocks (the other road is recorded 2.5 minutes later). The cost is recall: for cameras without a confident match the user sees the closest candidates.

4.4 Upload test pack. Seven 20–22 s clips (two sets of three Orchard Road cameras, one Bukit Timah Road camera) processed as uploads with *Fast processing* take about 1 s per second of video on a laptop CPU. Against the same pseudo ground truth, 71% (set 1) and 75% (set 2) of the cameras reported are correct and none is placed on Bukit Timah Road.

5. Findings and Discussion

- **Published models are strong on their own data.** FastReID reproduces its benchmark result, but on street footage with small, distant vehicles and new angles, Rank-1 falls from 97% to 36%.
- **The decision rule matters as much as the model.** A threshold alone placed 73% of queries on the distractor road; the ratio test and real-world time made the reported cameras mostly right, with no training.
- **Context makes results usable.** Location, real-world time, time between cameras and a frame per sighting turn scores into evidence a person can check, and ambiguous cases are shown as candidates rather than decided.
- **What changed during the project.** The first version trained its own matcher (LDA metric and Random Forest on generic embeddings); the published FastReID model replaced it. Timestamps in video time were replaced by real-world time, so cameras recorded minutes or hours apart can be compared.

Limitations. The demo and test cameras are zones of one recording, and their Singapore locations and times are curated; plates are rarely readable at street distance; zero-shot colour and type confuse similar classes (silver/grey/white; estate/hatchback/MPV); briefly seen vehicles may be dropped, especially with *Fast processing*; and the pseudo ground truth inherits the reference tracker's errors.

Future work. Test on separate physical cameras (e.g. CityFlow) and learn travel times between locations instead of one fixed window; add a plate detector before OCR; read camera locations and clocks from the camera system; process live streams.

6. Conclusion

V-Trace shows that a useful vehicle search and re-identification platform can be assembled from published pretrained models. The Re-ID model reproduces its published accuracy (mAP 81.6%), and on real street footage a cautious decision rule with real-world time makes 81% of the reported cameras correct, while locations, times and frames let a person check every result.

References

- Liu, X., Liu, W., Mei, T., & Ma, H. (2016). A Deep Learning-Based Approach to Progressive Vehicle Re-identification for Urban Surveillance (VeRi-776). ECCV.
- He, L., Liao, X., Liu, W., Liu, X., Cheng, P., & Mei, T. (2020). FastReID: A Pytorch Toolbox for General Instance Re-identification. arXiv:2006.02631.
- Pan, X., Luo, P., Shi, J., & Tang, X. (2018). Two at Once: Enhancing Learning and Generalization Capacities via IBN-Net. ECCV.
- Zhang, Y. et al. (2022). ByteTrack: Multi-Object Tracking by Associating Every Detection Box. ECCV.
- Radford, A. et al. (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP). ICML.
- Lowe, D. G. (2004). Distinctive Image Features from Scale-Invariant Keypoints. IJCV.
- Tang, Z. et al. (2019). CityFlow: A City-Scale Benchmark for Multi-Target Multi-Camera Vehicle Tracking and Re-Identification. CVPR.
- Jocher, G. et al. Ultralytics YOLO11 documentation. https://docs.ultralytics.com/ · JaidedAI. EasyOCR. https://github.com/JaidedAI/EasyOCR
- Sounds of Changes. Road traffic in Ljubljana (videos), Wikimedia Commons, CC BY 3.0.

Appendix – Running the System and AI Declaration

Install Python 3.11, run `pip install -r requirements.txt`, copy `.streamlit/secrets.toml.example` to `.streamlit/secrets.toml` with an admin username and password, then `streamlit run app.py` and open http://localhost:8501. Demo scenarios and search combinations with expected results are in the Demo Guide; the hosted link and assessor sign-in are provided with the submission. Benchmark and demo evaluation: `python scripts/eval_veri.py` and `python scripts/demo_eval.py`.

AI declaration. AI-based tools including Claude, ChatGPT and Gemini were used in a supporting role (drafting and refining text, organising content, coding assistance whose output was reviewed and tested, and research support). All ideas, analysis, technical design decisions and conclusions are the work of the project team.